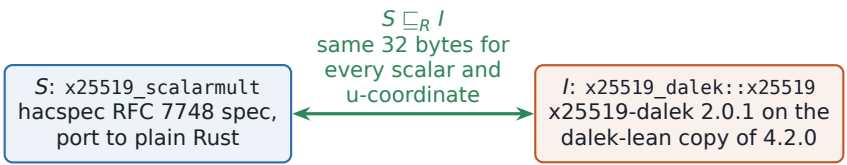
Full refinement of X25519

curve25519-dalek against the hacspec RFC 7748 spec, proved in Lean through Aeneas

ref-impl-simulation, track B

November 2026



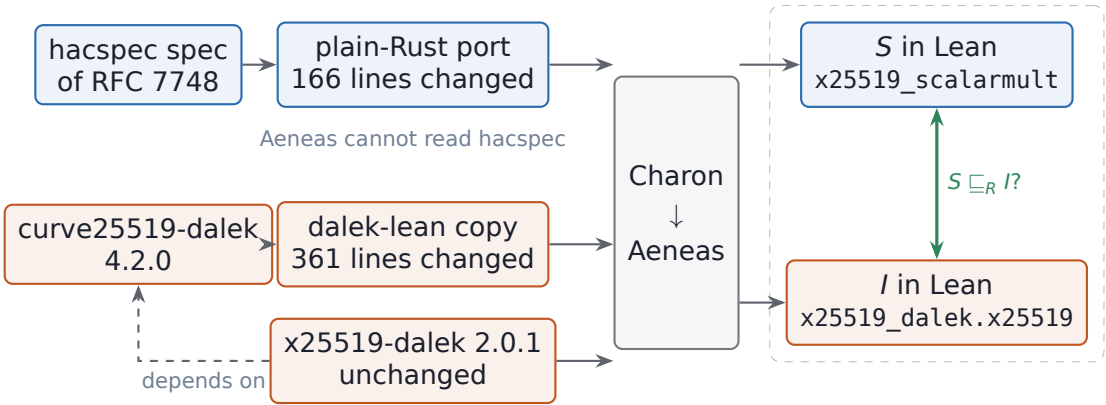
X25519 as an instance of the method

The method deck (`slides/methodology`) gives the general setup. This deck fills in each part for RFC 7748.

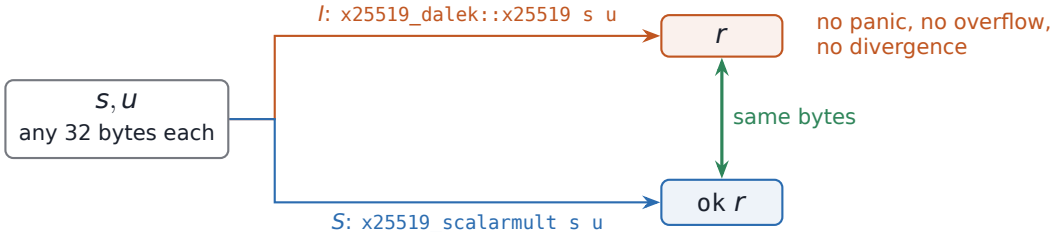
part of the method	X25519
spec S	hacspec spec of RFC 7748, ported to plain Rust: `x25519_scalarmult`
impl I	x25519-dalek 2.0.1 and the dalek-lean copy: `x25519_dalek::x25519`
R_{in}	the same scalar and u-coordinate on both sides, no premises
R_{out}	equal 32 bytes
data relation	5 limbs of 51 bits map to $\mathbb{Z}/p$, limbs below 2^{52}
lockstep	the ladder step: 19 field operations
hand-written	one formula (ring identity), the ladder loop (invariant up to a factor)
scope	full refinement: $S \sqsubseteq_R I$ for `x25519_dalek::x25519`

The question

Two Rust programs compute X25519: the spec S and the impl I. Aeneas turns both into Lean functions. Can Lean prove $S \sqsubseteq_R I$?



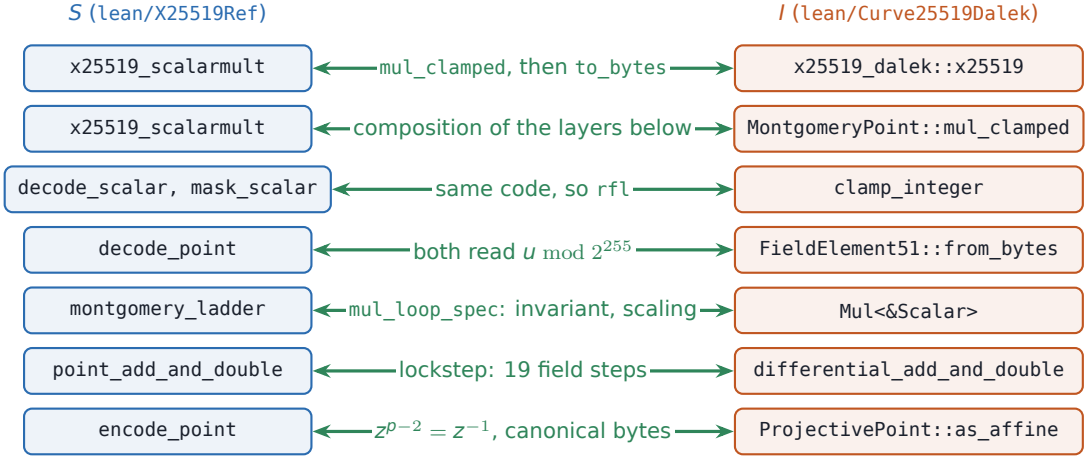
What $S \sqsubseteq_R I$ means here




```
theorem x25519_refines (k u : Array U8 32#usize) :
    x25519_dalek.x25519.x25519 k u ⦃ r =>
      x25519_ref.x25519_scalarmult k u = ok r ⦄
```

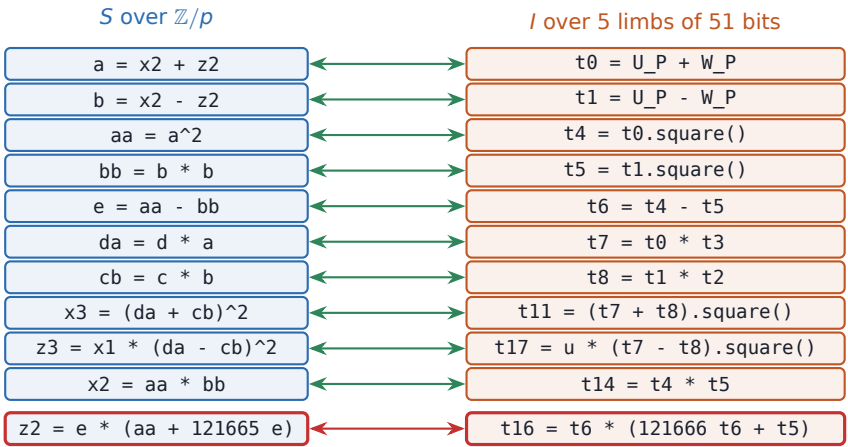
R_{in}: the same inputs. R_{out}: the same bytes. No premises. `#print axioms` shows only `propext`, `Classical.choice` and `Quot.sound`.

The proof follows the call path



Top row: `x25519_refines`, the theorem $S \sqsubseteq_R I$. Second row: `mul_clamped_refines`. Each other row is one layer.

The ladder step: lockstep on field operations

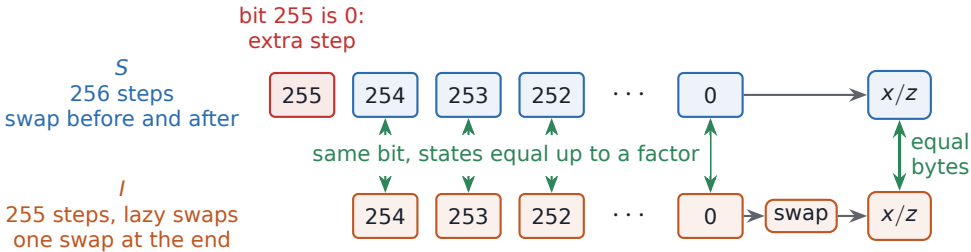


each rung

`toZ` maps the limbs to $\mathbb{Z}/p$, and the values are equal

the limbs stay below 2^{52}, so the next step can start

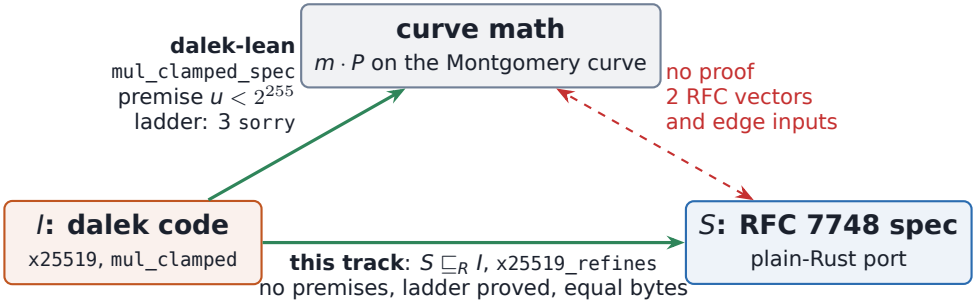
The ladder: the two loops do not line up



After the extra step, the state of S is the state of I with the second point times $4u$. Each later step scales each point by a nonzero factor.

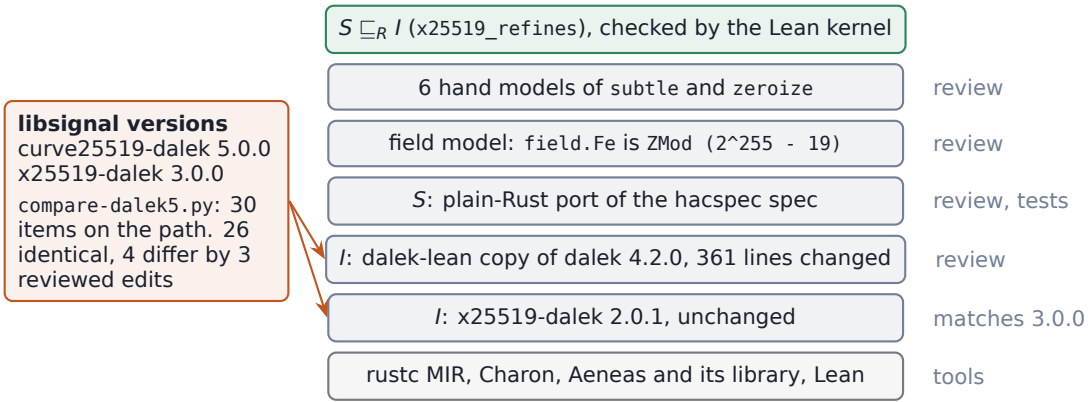
The encoding x/z cancels the factor (`enc_ladder_extra_step`). If $u = 0$, both outputs are 0. This algebra holds over any ring.

How this complements dalek-lean



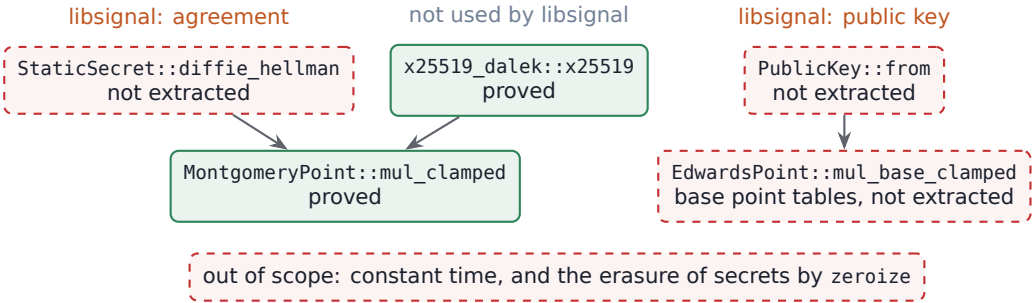
shared: the field specs of dalek-lean, ported to our Aeneas version:
`add sub mul square from_bytes to_bytes invert`
new here: `mul_loop_spec`, the ladder invariant

What the theorem trusts



Serial u64 backend only. Not `fiat`, not `curve25519_dalek_bits = "32"`.

What is not covered, and next steps



Next

Extract `diffie_hellman`.
Relate *S* to the curve math.
Build a lockstep tactic for the step.

Lesson

Lockstep closes the step.
The ladder needs an invariant by hand.